

Projeto de Compilador — Especificação Completa

INF01083 — Linguagens de Programação II / Compiladores — 2026/2

Prof. Leandro Wives — INF/UFRGS UFRGS,

Agosto–Novembro de 2026

Sumário

1	Visão Geral do Projeto	3
1.1	O que é o projeto	3
1.2	Calendário.....	4
1.3	Grupos e entrega	4
2	A Linguagem LARA	4
2.1	Descrição geral.....	4
2.2	Diferenças em relação ao C.....	5
2.3	Tipos de dados.....	5
2.4	Gramática (estilo Yacc).....	5
2.5	Precedência de operadores.....	8
2.6	Tokens.....	9
2.7	Exemplo de programa completo em LARA.....	10
3	Estrutura do Código Fornecido	10
3.1	Organização em diretórios.....	10
3.2	Princípio de acumulação	12
3.3	Como compilar	12
3.4	Como usar o compilador	12
3.5	Arquivos que os grupos NÃO devem modificar	12
4	Etapa 1 — Frontend do Compilador	13
4.1	Objetivo	13
4.2	O que o compilador faz nesta etapa.....	13
4.3	Arquivos e responsabilidades.....	13
4.4	Como o Compilador é Construído: Flex, Bison e o Pipeline.....	13
4.4.1	Visão geral do pipeline de compilação.....	13
4.4.2	Flex — o scanner	14
4.4.3	Bison — o parser	14
4.4.4	Como main()orquestra tudo — src/main.c	15
4.4.5	Guia de leitura do código: por onde começar	15
4.5	Tarefas detalhadas.....	16
4.5.1	TODO-A — Espaço em branco com tab (scanner.l).....	16
4.5.2	TODO-B — Float com expoente (scanner.l).....	16

4.5.3	TODO-C — Operadores &&e (parser.y)	16
4.5.4	TODO-D — Lista de argumentos com vírgula (parser.y)	17
4.5.5	TODO-E — ast_count_nodescom next(ast_walk.c)	17
4.5.6	TODO-F — ast_count_leaves(ast_walk.c)	17
4.5.7	TODO-G — ast_max_depth(ast_walk.c)	17
4.6	Aulas Relacionadas	18
4.7	Como Começar — Passo a Passo (Etapa 1)	18
5	Etapa 2 — Geração de TAC Básico	19
5.1	Objetivo	19
5.2	Representação TAC	19
5.2.1	Formato de impressão	19
5.2.2	Operações desta etapa	19
5.2.3	Temporários e nomes de endereços	20
5.2.4	Cálculo de offsets na tabela de símbolos	20
5.3	Exemplo completo	20
5.4	Tarefas dos alunos	21
5.4.1	TODO-E2-A — codegen_program(): variáveis globais (codegen.c)	21
5.4.2	TODO-E2-B — codegen_expr(): operadores aritméticos (codegen.c)	21
5.4.3	TODO-E2-C — codegen_expr(): operadores relacionais (codegen.c)	21
5.4.4	TODO-E2-D — codegen_stmt(): atribuição simples :=(codegen.c)	21
5.5	Aulas Relacionadas	21
5.6	Testando a Etapa 2	22
6	Etapa 3 — TAC Completo	22
6.1	Objetivo	22
6.2	Back-patching	22
6.2.1	API de back-patching (fornecida completa em backpatch.h)	23
6.3	Tradução de expressões booleanas com curto-circuito	23
6.4	Tradução de estruturas de controle	24
6.4.1	if (cond) { then } else { else }	24
6.4.2	while (cond) do { body }	24
6.4.3	for (init; cond; step) { body }	24
6.5	Geração de rótulos	24
6.6	Exemplo completo	24
6.7	Tarefas dos alunos	25
6.7.1	TODO-E3-A — codegen_bool_expr(): operadores &&e (codegen.c)	25
6.7.2	TODO-E3-B — codegen_stmt(): comando if/else (codegen.c)	26
6.7.3	TODO-E3-C — codegen_stmt(): comando while (codegen.c)	26
6.7.4	TODO-E3-D — codegen_stmt(): comando for (codegen.c)	26
6.8	Aulas Relacionadas	26
6.9	Testando a Etapa 3	26
7	Etapa 4 — Geração de Assembly x86-64	26
7.1	Objetivo	26
7.2	Convenções de chamada (System V ABI x86-64)	27
7.2.1	Prólogo padrão de função	27
7.2.2	Epílogo padrão de função	27

7.3	Alocação de variáveis e temporários.....	27
7.4	Mapeamento TAC → Assembly	27
7.4.1	Operações aritméticas.....	27
7.4.2	Operadores relacionais	28
7.4.3	Desvios e rótulos	28
7.4.4	Chamada de função.....	28
7.4.5	I/O (printf / scanf)	28
7.5	Otimizações peephole.....	28
7.6	Exemplo end-to-end	29
7.7	Tarefas dos alunos.....	29
7.7.1	TODO-E4-A — Expressões aritméticas e relacionais (asmgen.c)	29
7.7.2	TODO-E4-B — Convenção de chamada (asmgen.c)	29
7.7.3	TODO-E4-C — Otimizações peephole (peephole.c)	29
7.7.4	TODO-E4-D — Otimização sobre o TAC (bônus).....	29
7.8	Aulas Relacionadas.....	30
7.9	Testando a Etapa 4.....	30
8	Notas, Avaliação e Entrega	30
8.1	Colaboração com GitHub	30
8.1.1	Configuração inicial (uma única vez por grupo)	31
8.1.2	Fluxo de trabalho diário.....	31
8.1.3	Submissão no Moodle	32
8.2	Forma de entrega e prazos	32
8.2.1	Sessão de laboratório	32
8.2.2	Janela de 48 horas.....	32
8.2.3	Critério de aceite técnico.....	32
8.3	Critérios de avaliação.....	33
8.3.1	Etapa 1.....	33
8.3.2	Etapa 2.....	33
8.3.3	Etapa 3.....	33
8.3.4	Etapa 4.....	33
8.4	Notas sobre corretude e estilo	33
8.5	Dicas e armadilhas comuns.....	34
8.6	Recursos e referências	34

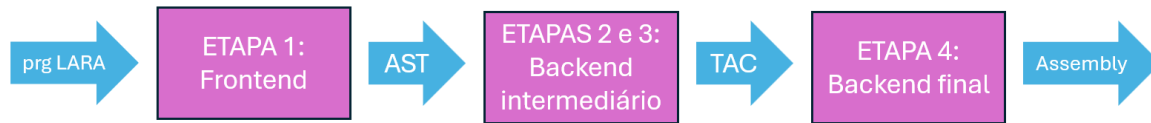
ATENÇÃO: Ambiente de desenvolvimento obrigatório – Todo o desenvolvimento e os testes do projeto devem ser realizados em **Linux** (Ubuntu 22.04 LTS ou equivalente). O compilador a ser desenvolvido usa GCC, flex e bison para a arquitetura x86-64 Linux. Submissões que funcionem apenas em macOS ou em Windows **não** serão aceitas.

1 Visão Geral do Projeto

1.1 O que é o projeto

O projeto da disciplina INF01083 consiste na construção incremental de um **compilador completo** para a linguagem **LARA** (*Linguagem Acadêmica Reduzida e Adaptada*), uma linguagem imperativa de pequeno porte.

O compilador é desenvolvido em **quatro etapas** ao longo do semestre, cada uma adicionando uma camada ao pipeline de compilação:



A entrada é um programa na linguagem LARA. A Etapa 1 corresponde ao Frontend do compilador, com um scanner gerado pelo flex e um parser gerado pelo bison, que produz uma AST (Árvore Sintática Abstrata).

As Etapas 2 e 3 correspondem à implementação de um Backend intermediário, que produz código intermediário abstrato (TAC) de três endereços.

Finalmente, a Etapa 4 corresponde à implementação de um Backend final, que monta código Assembly x86-64, o qual pode ser transformado em binário executável pelo GCC (gcc -no-pie).

As etapas e entregas correspondentes devem ser realizadas conforme o calendário a seguir.

1.2 Calendário

Etapas	Laboratório	Entrega	Tema
1	26 ago. 2026	26 ago. 2026	Frontend: scanner + parser + AST
2	30 set. 2026	30 set. 2026	Geração de TAC: expressões e declarações
3	26 out. 2026	26 out. 2026	TAC completo: controle de fluxo e funções
4	18 nov. 2026	18 nov. 2026	Assembly x86-64 e otimizações

1.3 Grupos e entrega

- O projeto é desenvolvido em **grupos de até 3 estudantes**.
- A entrega é feita via **Moodle** da disciplina, no formato .tar.gz.
- Cada entrega é apresentada ao professor no próprio laboratório.

2 A Linguagem LARA

2.1 Descrição geral

LARA é a sigla de *Linguagem Acadêmica Reduzida e Adaptada*. O nome reflete as três propriedades essenciais da linguagem: é *acadêmica* (projetada para ensino de compiladores), *reduzida* (subconjunto deliberadamente pequeno de C) e *adaptada* (sintaxe propositalmente diferente da linguagem-fonte de inspiração, de modo a exigir que os alunos leiam e compreendam o código fornecido).

LARA suporta tipos escalares, arrays unidimensionais, funções com parâmetros e retorno, estruturas de controle de fluxo e operações simples de entrada/saída.

2.2 Diferenças em relação ao C

Construção	C padrão	LARA
Declaração de variável	<code>int x = 5;</code>	<code>let x := 5;</code>
Declaração de array	<code>int v[10];</code>	<code>array 10 of int v;</code>
Cabeçalho de função	<code>int f(int a) { }</code>	<code>fun int f(int a) { }</code>
Atribuição	<code>x = expr;</code>	<code>x := expr;</code>
Laço while	<code>while (c) { }</code>	<code>while (c) do { }</code>
Literais booleanos	<code>1 / 0</code>	<code>true / false</code>

2.3 Tipos de dados

Tipo	Descrição	Tamanho
<code>int</code>	Inteiro com sinal (64 bits no runtime)	4 bytes
<code>float</code>	Ponto flutuante de dupla precisão	8 bytes
<code>char</code>	Caractere ASCII	1 byte
<code>bool</code>	Booleano (true/false)	1 byte
<code>void</code>	Ausência de valor (apenas em funções)	—

2.4 Gramática (estilo Yacc)

```
program
  : toplevel_list
  | /* vazio */
  ;

toplevel_list
  : toplevel_decl
  | toplevel_list toplevel_decl
  ;

toplevel_decl
  : fun_decl
  | var_decl
  | array_decl
  ;

fun_decl
  : 'fun' type_spec ID '(' param_list ')' block
  ;

param_list
  : /* vazio */
  | param_list_ne
  ;
```

```

param_list_ne
    : param
    | param_list_ne ',' param
    ;

param
    : type_spec ID
    ;

var_decl
    : type_spec ID ';'
    ;

array_decl
    : 'array' INT_LIT 'of' type_spec ID ';'
    ;

type_spec
    : 'int'
    | 'float'
    | 'bool'
    | 'char'
    | 'void'
    ;

block
    : '{' stmt_list '}'
    | '{' '}'
    ;

stmt_list
    : stmt
    | stmt_list stmt
    ;

stmt
    : simple_stmt ';'
    | compound_stmt
    ;

simple_stmt
    : assign_stmt
    | return_stmt
    | print_stmt
    | read_stmt
    | call_stmt
    | var_local
    ;

```

```

compound_stmt
    : if_stmt
    | while_stmt
    | for_stmt
    ;

var_local
    : 'let' ID ':' '=' expr
    ;

assign_stmt
    : lvalue ':' '=' expr
    | lvalue '+' '=' expr
    | lvalue '-' '=' expr
    ;

if_stmt
    : 'if' '(' expr ')' block
    | 'if' '(' expr ')' block 'else' block
    ;

while_stmt
    : 'while' '(' expr ')' 'do' block
    ;

for_stmt
    : 'for' '(' assign_stmt ';' expr ';' assign_stmt ')' block
    ;

return_stmt
    : 'return' expr
    ;

print_stmt
    : 'print' expr
    ;

read_stmt
    : 'read' lvalue
    ;

call_stmt
    : ID '(' expr_list ')'
    ;

expr
    : expr '+' expr

```

```

| expr '-' expr
| expr '*' expr
| expr '/' expr
| expr '%' expr
| expr '<' expr
| expr '>' expr
| expr '<=' expr
| expr '>=' expr
| expr '==' expr
| expr '!=' expr
| expr '&&' expr
| expr '||' expr
| '!' expr
| '-' expr
| '(' expr ')'
| ID '(' expr_list ')'
| ID '[' expr ']'
| ID
| literal
;

expr_list
: /* vaziao */
| expr_list_ne
;

expr_list_ne
: expr
| expr_list_ne ',' expr
;

lvalue
: ID
| ID '[' expr ']'
;

literal
: INT_LIT
| FLOAT_LIT
| CHAR_LIT
| STRING_LIT
| 'true'
| 'false'
;

```

2.5 Precedência de operadores

Nível	Operadores	Associatividade
1	:=, +=, -=	direita
2		esquerda
3	&&	esquerda
4	==, !=	esquerda
5	<, >, <=, >=	esquerda
6	+, -	esquerda
7	*, /, %	esquerda
8	!, - (unário)	direita
9	[	esquerda

2.6 Tokens

Token	Valor	Descrição
TK_PR_INT	258	int
TK_PR_FLOAT	259	float
TK_PR_BOOL	260	bool
TK_PR_CHAR	261	char
TK_PR_IF	262	if
TK_PR_ELSE	263	else
TK_PR_WHILE	264	while
TK_PR_FOR	265	for
TK_PR_RETURN	266	return
TK_PR_VOID	267	void
TK_PR_ARRAY	268	array
TK_PR_OF	269	of
TK_PR_LET	270	let
TK_PR_FUN	271	fun
TK_PR_DO	272	do
TK_PR_PRINT	273	print
TK_PR_READ	274	read
TK_LIT_TRUE	275	true
TK_LIT_FALSE	276	false
TK_OC_LE	277	<=
TK_OC_GE	278	>=
TK_OC_EQ	279	==
TK_OC_NE	280	!=
TK_OC_AND	281	&&
TK_OC_OR	282	\\
TK_OC_ASSIGN	284	:=
TK_OC_PLUSEQ	285	+=
TK_OC_MINUSEQ	286	-=
TK_ERROR	287	Caractere inválido (erro léxico)
TK_ID	288	Identificador
TK_LIT_INT	289	Literal inteiro (42, 0)
TK_LIT_FLOAT	290	Literal float (3.14, 1.5e10)

Token	Valor	Descrição
TK_LIT_CHAR	291	Literal char ('a')
TK_LIT_STRING	292	Literal string ("texto")

Caracteres simples (+, -, *, /, %, <, >, !, =, (,), {, }, [,], ,, ;, :) são retornados pelo scanner com seus respectivos valores ASCII.

2.7 Exemplo de programa completo em LARA

```
/* Calcula o fatorial de um número */
fun int fatorial(int n) {
  if (n <= 1) {
    return 1 ;
  } else {
    let r := fatorial(n - 1) ;
    return n * r ;
  }
}

fun void main() {
  let f := fatorial(5) ;
  print f ;
}
```

3 Estrutura do Código Fornecido

3.1 Organização em diretórios

O projeto é distribuído em quatro diretórios independentes, um por etapa:

```
projeto/
|-- etapal/
|   |-- Makefile
|   |-- README.md
|   |-- doc/
|   |   `-- especificacao_etapal.pdf
|   |-- src/
|   |   |-- tokens.h      ← completo
|   |   |-- ast.h / ast.c ← completos
|   |   |-- symlab.h / symlab.c ← completos
|   |   |-- main.c       ← completo
|   |   |-- scanner.l    ← ESQUELETO (TODOs A e B)
|   |   |-- parser.y     ← ESQUELETO (TODOs C e D)
|   |   `-- ast_walk.c   ← ESQUELETO (TODOs E, F e G)
|   `-- tests/
|       |-- valid/      ← programas LARA válidos
```

```

|         |-- invalid/          ← programas com erros
|         `-- run_tests.sh
|
|-- etapa2/
|   |-- Makefile
|   |-- README.md
|   |-- doc/
|   |   `-- especificacao_etapa2.pdf
|   |-- src/
|   |   |-- (tudo da E1, já completo)
|   |   |-- tac.h / tac.c      ← completos (não modificar)
|   |   |-- codegen.h         ← completo
|   |   `-- codegen.c         ← ESQUELETO (TODOs E2-A a E2-D)
|   `-- tests/
|       |-- valid/*.lc + *.expected
|       |-- invalid/
|       `-- run_tests.sh
|
|-- etapa3/
|   |-- Makefile
|   |-- README.md
|   |-- doc/
|   |   `-- especificacao_etapa3.pdf
|   |-- src/
|   |   |-- (tudo da E2, já completo)
|   |   |-- backpatch.h / backpatch.c ← completos (não modificar)
|   |   `-- codegen.c          ← ESQUELETO (TODOs E3-A a E3-D)
|   `-- tests/
|       |-- valid/*.lc + *.expected
|       |-- invalid/
|       `-- run_tests.sh
|
`-- etapa4/
    |-- Makefile
    |-- README.md
    |-- doc/
    |   `-- especificacao_etapa4.pdf
    |-- src/
    |   |-- (tudo da E3, já completo)
    |   |-- asmggen.h / asmggen.c ← ESQUELETO (TODOs E4-A e E4-B)
    |   |-- peephole.h / peephole.c ← ESQUELETO (TODO E4-C)
    |   `-- main.c                ← atualizado para pipeline completo
    `-- tests/
        |-- valid/*.lc + *.expected
        |-- invalid/
        `-- run_tests.sh

```

3.2 Princípio de acumulação

Cada etapa **parte da solução completa da etapa anterior**. O diretório `src/` de cada etapa já contém todos os módulos funcionais das etapas anteriores — os grupos não precisam copiar arquivos manualmente. Só os arquivos marcados como **ESQUELETO** precisam ser completados.

3.3 Como compilar

Em qualquer etapa:

```
cd etapaN/  
make          # compila tudo  
make test     # compila e executa os testes  
make clean    # remove binários e arquivos gerados
```

O binário resultante chama-se `lara` em todas as etapas.

3.4 Como usar o compilador

```
# O compilador lê da entrada padrão  
./lara < programa.lc  
  
# Ou com redireção de arquivo  
./lara < programa.lc > saida.tac # (etapas 2 e 3)  
./lara < programa.lc > programa.s # (etapa 4)  
  
# Pipeline completo (etapa 4)  
./lara < programa.lara > programa.s  
gcc programa.s -o programa -no-pie  
./programa
```

Código de saída:

- 0 — compilação bem-sucedida
- 1 — erro léxico ou sintático

3.5 Arquivos que os grupos NÃO devem modificar

Os seguintes arquivos são infraestrutura fornecida completa. Modificá-los pode quebrar a compatibilidade com os testes automatizados:

Arquivo	Fornecido em	O que contém
<code>tokens.h</code>	E1	Valores numéricos dos tokens
<code>ast.h</code> / <code>ast.c</code>	E1	Estrutura e operações da AST
<code>tac.h</code> / <code>tac.c</code>	E2	Estrutura TAC, impressão, geração de temp
<code>backpatch.h/c</code>	E3	Listas de saltos pendentes (back-patch)

4 Etapa 1 — Frontend do Compilador

Laboratório: 26 de agosto de 2026

4.1 Objetivo

Revisar e completar o frontend do compilador: scanner (análise léxica via flex), parser (análise sintática via bison) e percorrimento da AST resultante.

Os grupos recebem um compilador quase completo e devem preencher **7 pontos intencionalmente deixados em aberto**, marcados TODO-A a TODO-G.

4.2 O que o compilador faz nesta etapa

Ao receber um programa LARA válido, o compilador da Etapa 1:

1. Executa a análise léxica e sintática
2. Constrói a AST do programa
3. Imprime a AST em formato indentado (na stdout)
4. Imprime estatísticas da AST (número de nós, folhas, profundidade)
5. Imprime a tabela de símbolos

Em caso de erro léxico ou sintático, imprime uma mensagem no `stderr` e retorna o código 1.

4.3 Arquivos e responsabilidades

Arquivo	Status	O que os alunos fazem
tokens.h	Completo	Apenas consultar
ast.h/ast.c	Completo	Apenas usar as funções
symtab.h/c	Completo	Apenas usar as funções
main.c	Completo	Apenas ler para entender o fluxo
scanner.l	Esqueleto	Completar TODO-A e TODO-B
parser.y	Esqueleto	Completar TODO-C e TODO-D
ast_walk.c	Esqueleto	Implementar TODO-E, TODO-F e TODO-G

4.4 Como o Compilador é Construído: Flex, Bison e o Pipeline

Esta seção explica o que cada ferramenta faz, quais arquivos produz e como as peças se encaixam para compor o compilador LARA. Leia com atenção antes de começar a editar qualquer arquivo.

4.4.1 Visão geral do pipeline de compilação

Compilar o projeto com `make` executa os seguintes passos, nesta ordem:

```
src/scanner.l --flex--> src/lex.yy.c      (scanner em C)
src/parser.y  --bison--> src/parser.tab.c  (parser em C)
                                   src/parser.tab.h  (definicao dos tokens)

src/lex.yy.c  --|
src/parser.tab.c --|
```

```
src/ast.c      --|-- gcc --> lara      (binario final)
src/symtab.c   --|
src/ast_walk.c --|
src/main.c     --|
```

4.4.2 Flex — o scanner

O que é: Flex (*Fast Lexical Analyzer Generator*) é uma ferramenta que gera scanners automaticamente a partir de expressões regulares.

Entrada — src/scanner.l: dividido em três seções separadas por %%:

```
definições      <- macros como DIGIT, LETTER, FLOAT
%%
regras          <- pares (expressão regular) { ação em C }
%%
código auxiliar <- funções C extras (normalmente vazio)
```

Cada regra tem a forma `expressao_regular { código C }`. Exemplos:

```
"if"    { return TK_PR_IF; }          /* palavra reservada */

{ID}    { yylval.sym = symtab_insert(global_symtab, yytext, yylineno);
        return TK_ID; }              /* identificador */
```

`yytext` contém o texto do token reconhecido; `yylineno`, o número de linha corrente.

Comando de invocação:

```
flex -o src/lex.yy.c src/scanner.l
```

Saída — src/lex.yy.c: contém a função `int yylex(void)`. Cada chamada lê a entrada, avança até o fim do próximo token e **retorna seu código numérico** (ex.: `TK_PR_IF`, `TK_LIT_INT`). Se o token tiver valor associado (identificador ou literal), ele é armazenado em `yylval` antes de retornar. `yylex()` é chamada repetidamente pelo parser — os alunos nunca precisam chamá-la diretamente.

4.4.3 Bison — o parser

O que é: Bison é um gerador de parsers LALR(1). Recebe uma gramática com ações semânticas em C e gera automaticamente o respectivo parser.

Entrada — src/parser.y: também dividido em três seções por %%:

```
declaracoes    <- tokens (%token), tipos (%type), precedencia (%left, %right)
%%
gramatica      <- regras de producao com acoes semanticas em C
%%
auxiliar       <- funcoes C extras (ex.: yyerror)
```

Cada regra tem a forma:

```
nao_terminal
: sequencia_de_simbolos { acao semantica em C }
| outra_alternativa     { acao semantica em C }
;
```

`$$` é o valor do não-terminal sendo reduzido; `$1`, `$2`, ... são os valores dos símbolos do lado direito. Exemplo que constrói um nó de soma na AST:

```
expr
: expr '+' expr
  { $$ = ast_new(AST_EXPR_BINARY, "+", yylineno);
    $$->children[0] = $1;
    $$->children[1] = $3; }
;
```

Comando de invocação:

```
bison -d -o src/parser.tab.c src/parser.y
```

Saída — `src/parser.tab.c` e `src/parser.tab.h`:

- `parser.tab.c` contém a função `int yyparse(void)`, que implementa o autômato LALR(1). Ela chama `yylex()` repetidamente, executa as ações semânticas das reduções e retorna 0 em caso de sucesso ou 1 em caso de erro sintático.
- `parser.tab.h` define os valores numéricos de todos os tokens declarados com `%token`. É incluído pelo scanner (`#include "parser.tab.h"`) para que `yylex()` e `yyparse()` usem os mesmos códigos numéricos.

4.4.4 Como `main()` orquestra tudo — `src/main.c`

`main.c` é o ponto de entrada do compilador. Ele não faz análise léxica nem sintática diretamente — delega essas tarefas às funções geradas pelo flex e pelo bison:

```
int main(int argc, char *argv[]) {
    global_syntab = syntab_new();           // 1. tabela de simbolos global
    int resultado = yyparse();               // 2. analise lexica + sintatica + AST
    if (resultado == 0) {
        ast_print(ast_root, 0, stdout);     // 3. imprime a AST
        main_walk(ast_root);                // 4. percorre e imprime estatisticas
        syntab_print(global_syntab, stdout);
    }
    ast_free(ast_root);
    syntab_free(global_syntab);
    return resultado;
}
```

Por que `yyparse()` dá acesso à AST? A variável global `ast_root` é declarada em `parser.y` e exposta como extern em `main.c`. Durante a análise, quando o parser reconhece o programa inteiro (reduz para o símbolo inicial `program`), executa a ação:

```
program : toplevel_list { ast_root = $1; }
```

Assim, ao retornar de `yyparse()`, `ast_root` já aponta para a raiz da AST completa — pronta para ser percorrida por `main_walk()` ou pelas etapas seguintes do compilador.

4.4.5 Guia de leitura do código: por onde começar

Explore os arquivos nesta ordem:

1. **src/tokens.h** — lista de constantes dos tokens (apenas referência).
 2. **src/ast.h** — tipo `ast_node_t`; funções `ast_new()`, `ast_add_child()`, `ast_append()`, `ast_print()`. Entenda os campos `type`, `value`, `children[]` e `next` — aparecem em todos os outros módulos.
 3. **src/symtab.h** — interface da tabela de símbolos; `symtab_insert()` e `symtab_lookup()`.
 4. **src/main.c** — ponto de entrada; leia na íntegra (é curto).
 5. **src/scanner.l** — especificação léxica; localize os TODOs A e B.
 6. **src/parser.y** — gramática com ações semânticas; observe como cada regra cria nós AST; localize os TODOs C e D.
 7. **src/ast_walk.c** — funções de percorrimento; implemente os TODOs E, F e G.
-

4.5 Tarefas detalhadas

4.5.1 TODO-A — Espaço em branco com tab (scanner.l)

Problema: a regra atual ignora apenas espaço () e newline (\n). Programas com tabulação (\t) provocam erro léxico.

Tarefa: estender a expressão regular para incluir \t e \r.

```
/* Antes */
[ \n]+      { /* ignora */ }
```

```
/* Depois */
[ \n\t\r]+  { /* ignora */ }
```

4.5.2 TODO-B — Float com expoente (scanner.l)

Problema: a definição atual não reconhece notação científica (1.5e10, 2.3E-4).

Tarefa: estender o regex `FLOAT` para suportar expoente opcional.

```
/* Antes */
FLOAT  {DIGIT}*"."{DIGIT}+|{DIGIT}+"."{DIGIT}*

/* Depois */
FLOAT  ({DIGIT}*"."{DIGIT}+|{DIGIT}+"."{DIGIT}*) ([eE] [+ -]?{DIGIT}+)?
```

4.5.3 TODO-C — Operadores && e || (parser.y) Problema:

as ações semânticas para `&&` e `||` retornam `NULL`.

Tarefa: criar nós `AST_EXPR_BINARY` para os dois operadores lógicos, seguindo o padrão dos outros operadores binários.

```
/* Antes */
| expr TK_OC_AND expr  { $$ = NULL; }
| expr TK_OC_OR  expr  { $$ = NULL; }

/* Depois */
| expr TK_OC_AND expr  {
```

```

    $$ = ast_new(AST_EXPR_BINARY, "&&", yylineno);
    $$->children[0] = $1;  $$->children[1] = $3;
}
| expr TK_OC_OR expr {
    $$ = ast_new(AST_EXPR_BINARY, "||", yylineno);
    $$->children[0] = $1;  $$->children[1] = $3;
}

```

4.5.4 TODO-D — Lista de argumentos com vírgula (parser.y)

Problema: a regra `expr_list_ne` aceita apenas um argumento. Chamadas como `f(a,b,c)` falham.

Tarefa: adicionar a produção recursiva com `ast_append()`.

```

/* Antes */
expr_list_ne : expr { $$ = $1; }
              /* falta regra para lista com vírgula */
              ;

/* Depois */
expr_list_ne : expr { $$ = $1; }
              | expr_list_ne ',' expr { $$ = ast_append($1, $3); }
              ;

```

4.5.5 TODO-E — `ast_count_nodes` com `next` (ast_walk.c)

Problema: a função percorre os filhos (`children[]`) mas ignora o encadeamento `next`, subestimando a contagem em programas com múltiplos comandos.

Tarefa: adicionar a chamada recursiva via `next`:

```
count += ast_count_nodes(node->next);
```

4.5.6 TODO-F — `ast_count_leaves` (ast_walk.c)

Problema: a função sempre retorna 0.

Definição: um nó é folha quando todos os seus `children[]` são NULL e `next` também é NULL.

Tarefa: implementar a função com percurso recursivo (filhos e `next`).

4.5.7 TODO-G — `ast_max_depth` (ast_walk.c)

Problema: a função sempre retorna 0.

Definição: profundidade máxima da árvore — a raiz tem profundidade 0. O encadeamento `next` **não** aumenta a profundidade (mesmo nível).

Tarefa: implementar a função com percurso recursivo dos filhos.

4.6 Aulas Relacionadas

Os tópicos necessários para a Etapa 1 são cobertos nas cinco aulas que antecedem o laboratório de 26 de agosto de março:

Aula	Data	Tema	Relevância para E1
E1	05/08	Introdução geral — fases de compilador;	Contexto e estrutura do semestre scanner.l: TODOs A e B
E2	10/08	Análise léxica — tokens, expressões regulares, AFND→AFD, flex	
E3	12/08	Análise sintática descendente — GLCs, LL(1), estrutura de arquivo bison	parser.y: base teórica
E4	17/08	Análise ascendente — LALR(1), conflitos, ações semânticas no bison	parser.y: TODOs C e D
E5	19/08	AST — construção com ações semânticas, percorrimento recursivo em C; Visão geral do projeto e da LARA	ast.h/ast.c: TODOs E, F, G

4.7 Como Começar — Passo a Passo (Etapa 1)

1. Obter e descompactar o código (no Linux):

```
tar xzf etapa1-INF01083-2026-1.tar.gz
cd etapa1/
```

2. Instalar dependências (uma única vez):

```
sudo apt-get install -y build-essential flex bison
```

3. Compilar:

```
make
```

O binário lara é gerado no diretório etapa1/. Erros de compilação não são esperados — o scaffold já compila desde o início.

4. Teste rápido:

```
echo 'fun void main() { let x := 42 ; print x ; }' | ./lara
```

5. Executar a suite de testes:

```
make test
```

Com o scaffold inicial, os testes de programas **válidos falharão** (os TODOs ainda não estão implementados) — isso é esperado. Os testes de programas **inválidos devem passar**: o compilador já rejeita erros sintáticos.

6. Antes de editar qualquer arquivo, leia a seção “*Como o Compilador é Construído: Flex, Bison e o Pipeline*” desta especificação e siga o Guia de leitura sugerido nela.

5 Etapa 2 — Geração de TAC Básico

Laboratório: 30 de setembro de 2026

5.1 Objetivo

Implementar a geração de **Código de Três Endereços (TAC)** para as construções mais simples de LARA: expressões aritméticas e relacionais, declarações de variáveis com cálculo de offsets e atribuições.

Ao final desta etapa, o compilador recebe um programa LARA e **imprime na stdout o TAC correspondente**, uma instrução por linha.

5.2 Representação TAC

O TAC é uma sequência de **quádruplas** (op, result, arg1, arg2), impressas em formato legível. Cada instrução usa, no máximo, três “endereços” (variáveis, temporários ou literais).

5.2.1 Formato de impressão

```
beginFunc main      ← início de função
  local x, 0         ← declaração de variável local (offset 0)
  _t1 = 4 * 2         ← operação binária (temporário _t1)
  _t2 = 3 + _t1       ← expressão com temporário
  x = _t2            ← cópia (atribuição)
  print x            ← saída
endFunc main        ← fim de função
```

5.2.2 Operações desta etapa

Instrução TAC	Formato impresso	Descrição
TAC_COPY	t=a	Cópia simples
TAC_ADD	t=a+b	Adição
TAC_SUB	t=a-b	Subtração
TAC_MUL	t=a*b	Multiplicação
TAC_DIV	t=a/b	Divisão
TAC_MOD	t=a%b	Módulo
TAC_NEG	t = -a	Negação aritmética
TAC_LT	t = a<b	Menor que (resultado 0 ou 1)
TAC_GT	t = a >b	Maior que
TAC_LE	t = a <= b	Menor ou igual
TAC_GE	t = a >= b	Maior ou igual
TAC_EQ	t = a == b	Igualdade
TAC_NE	t = a != b	Diferença
TAC_LOAD	t=v[i]	Leitura de array
TAC_STORE	v[i]=t	Escrita em array
TAC_DECL_GLOBAL	global v, size	Declaração de variável global
TAC_DECL_LOCAL	local v, offset	Declaração de variável local
TAC_BEGINFUNC	beginFunc f	Início de função

Instrução TAC	Formato impresso	Descrição
TAC_ENDFUNC	endFunc f	Fim de função
TAC_PRINT	print a	Saída padrão
TAC_READ	read v	Leitura de entrada
TAC_RETURN	return a	Retorno com valor
TAC_RETURN_VOID	return	Retorno sem valor
TAC_PARAM	param a	Argumento de chamada
TAC_CALL	t = call f N	Chamada de função (N argumentos)

5.2.3 Temporários e nomes de endereços

1. **Temporários** gerados pelo compilador: `_t1`, `_t2`, `_t3`, ...
2. **Variáveis** do programa-fonte: usam seu nome original (`x`, `soma`, `v`)
3. **Literais** inteiros e floats: representados como strings ("`42`", "`3.14`")

5.2.4 Cálculo de offsets na tabela de símbolos

Cada variável recebe um **offset** em bytes, calculado a partir de zero:

1. **Variáveis globais**: alocadas sequencialmente a partir do offset 0 no segmento `.bss`
2. **Variáveis locais**: alocadas sequencialmente a partir do offset 0 no frame da função
3. **Tamanho por tipo**: `int=4`, `float=8`, `char=1`, `bool=1`. Exemplo: `inta;float b;int c;` → offsets 0, 4, 12.

A tabela de símbolos da Etapa 1 é estendida com dois campos novos:

Campo	Tipo	Descrição
scope	sym_scope_t	SYM_SCOPE_GLOBAL ou SYM_SCOPE_LOCAL
array_size	int	Número de elementos (apenas para arrays)

5.3 Exemplo completo

Entrada (01_soma.lc):

```
fun void main() {
    let x := 3 + 4 * 2 ;
    let y := x * x - 1 ;
    print y ;
}
```

Saída esperada:

```
beginFunc main
    local x, 0
    _t1 = 4 * 2
    _t2 = 3 + _t1
    x = _t2
    local y, 4
```

```

    _t3 = x * x
    _t4 = _t3 - 1
    y = _t4
    print y
endFunc main

```

5.4 Tarefas dos alunos

5.4.1 TODO-E2-A — codegen_program(): variáveis globais (codegen.c)

A função `codegen_program()` atualmente só processa funções. Complete o loop para processar também declarações globais de variáveis e arrays.

Para cada `AST_VAR_DECL` ou `AST_ARRAY_DECL` no topo do programa: 1. Busque a entrada na tabela de símbolos com `symtab_lookup()`; 2. Atribua `e->scope = SYM_SCOPE_GLOBAL` e `e->offset = global_offset`; 3. Incremente `global_offset` pelo tamanho do tipo (`xarray_size` para arrays); 4. Emita `TAC_DECL_GLOBAL` com o nome e o tamanho em bytes

5.4.2 TODO-E2-B — codegen_expr(): operadores aritméticos (codegen.c)

O caso `AST_EXPR_BINARY` usa `TAC_NOP` como placeholder. Substitua pela emissão correta com o opcode real:

```

/* Substitua: */
codegen_emit(ctx, TAC_NOP, tmp, left, right);

/* Por: */
codegen_emit(ctx, op, tmp, left, right);

```

A função `op_to_tac()` já converte a string do operador ("+", "-", etc.) para o opcode TAC correto.

5.4.3 TODO-E2-C — codegen_expr(): operadores relacionais (codegen.c)

Verifique se a correção do TODO-E2-B também cobre os operadores relacionais (<, >, <=, >=, ==, !=), uma vez que `op_to_tac()` já os mapeia.

5.4.4 TODO-E2-D — codegen_stmt(): atribuição simples := (codegen.c)

O caso `AST_ASSIGN` com operador `:=` emite apenas uma mensagem de aviso. Implemente:

1. Gere código para a expressão do lado direito: `codegen_expr(ctx, stmt->children[1])`
2. Obtenha o nome do lvalue em `stmt->children[0]->value` (para `AST_SYMBOL`)
3. Emita `TAC_COPY` do resultado para o lvalue
4. Libere a string retornada por `codegen_expr()` com `free()`

Para o caso bônus de lvalue indexado (`AST_EXPR_INDEX`), use `TAC_STORE`.

5.5 Aulas Relacionadas

Os tópicos necessários para a Etapa 2 são cobertos nas aulas entre os laboratórios de agosto e setembro.

5.6 Testando a Etapa 2

```
cd etapa2/  
make  
bash tests/run_tests.sh
```

O script compara a saída do compilador com os arquivos `.expected`. Os 5 testes válidos devem produzir um TAC exatamente igual ao esperado **após a resolução dos TODOs**. O scaffold (com TODOs em aberto) produz uma saída incompleta — isso é esperado.

6 Etapa 3 — TAC Completo

Laboratório: 26 de outubro de 2026

6.1 Objetivo

Estender o gerador de TAC da Etapa 2 para cobrir as construções restantes de LARA:

- **Expressões booleanas** com avaliação em curto-circuito (`&&`, `||`)
- **Controle de fluxo:** `if/else`, `while...do`, `for`
- **Chamadas de função** (já parcialmente implementadas via `TAC_CALL/TAC_PARAM`)

Ao final, o compilador deve processar programas completos em LARA, incluindo funções recursivas.

6.2 Back-patching

A técnica de **back-patching** resolve o problema de emitir saltos condicionais antes de saber o rótulo de destino. Funciona em duas fases:

Fase 1 — Emissão: emite o salto com rótulo placeholder `"???"` e registra a instrução em uma “lista de saltos pendentes” (`patch_list_t`).

Fase 2 — Preenchimento: quando o rótulo de destino é conhecido, percorre a lista e substitui `"???"` pelo rótulo real.

6.2.1 API de back-patching (fornecida completa em backpatch.h)

```
/* Cria lista com uma instrução pendente */
patch_list_t *patch_list_make(tac_instr_t *instr);

/* Une duas listas */
patch_list_t *patch_list_merge(patch_list_t *l1, patch_list_t *l2);

/* Preenche todos os saltos com 'label' */
void patch_list_backpatch(patch_list_t *list, const char *label);

/* Libera a lista (não as instruções TAC) */
void patch_list_free(patch_list_t *list);
```

A estrutura `bool_result_t` (definida em `backpatch.h`) encapsula o resultado de traduzir uma expressão booleana:

```
typedef struct {
    patch_list_t *true_list; /* saltos pendentes quando a expr é
                             verdadeira */
    patch_list_t *false_list; /* saltos pendentes quando a expr é
                              falsa */
} bool_result_t;
```

6.3 Tradução de expressões booleanas com curto-circuito

Para uma expressão atômica (relacional ou variável), emitem-se dois saltos pendentes:

```
[calcula expr → _tN]
ifTrue  _tN goto ??? ← entra em true_list
ifFalse _tN goto ??? ← entra em false_list
```

Para `e1&&e2`:

```
[código de e1]
ifTrue  e1 goto _Lmid ← true_list de e1 preenchida com _Lmid
ifFalse e1 goto ???   ← false_list de e1&&e2
_Lmid:
[código de e2]
ifTrue  e2 goto ??? ← true_list de e1&&e2
ifFalse e2 goto ??? ← mais elementos da false_list de e1&&e2
```

Para `e1||e2`:

```
[código de e1]
ifTrue  e1 goto ??? ← true_list de e1||e2
ifFalse e1 goto _Lmid ← false_list de e1 preenchida com _Lmid
_Lmid:
[código de e2]
ifTrue  e2 goto ??? ← mais elementos da true_list de e1||e2
ifFalse e2 goto ??? ← false_list de e1||e2
```

6.4 Tradução de estruturas de controle

6.4.1 if (cond) { then } else { else }

```
    [código de cond → true_list, false_list]
    backpatch(true_list, _Ltrue)
_Ltrue:
    [código de then]
    goto _Lend
    backpatch(false_list, _Lfalse)
_Lfalse:
    [código de else]
_Lend:
```

Sem else: suprime o gotoe _Lfalse torna-se o ponto de saída.

6.4.2 while (cond) do { body }

```
_Lbegin:
    [código de cond → true_list, false_list]
    backpatch(true_list, _Lbody)
_Lbody:
    [código de body]
    goto _Lbegin
    backpatch(false_list, _Lend)
_Lend:
```

6.4.3 for (init; cond; step) { body }

```
    [código de init]
_Lbegin:
    [código de cond → true_list, false_list]
    backpatch(true_list, _Lbody)
_Lbody:
    [código de body]
    [código de step]
    goto _Lbegin
    backpatch(false_list, _Lend)
_Lend:
```

6.5 Geração de rótulos

Use `tac_new_label()` (fornecido em `tac.h`) para gerar rótulos únicos: `_L1`, `_L2`, ..., incrementando automaticamente.

6.6 Exemplo completo

Entrada:

```
fun int fatorial(int n) {
    if (n <= 1) {
```

```

        return 1 ;
    } else {
        let r := fatorial(n - 1) ;
        return n * r ;
    }
}

fun void main() {
    let f := fatorial(5) ;
    print f ;
}

```

Saída esperada:

```

beginFunc fatorial
    _t1 = n <= 1
    ifTrue _t1 goto _L1
    ifFalse _t1 goto _L3
_L1:
    return 1
    goto _L2
_L3:
    local r, 0
    _t2 = n - 1
    param _t2
    _t3 = call fatorial 1
    r = _t3
    _t4 = n * r
    return _t4
_L2:
endFunc fatorial
beginFunc main
    local f, 0
    param 5
    _t5 = call fatorial 1
    f = _t5
    print f
endFunc main

```

6.7 Tarefas dos alunos

6.7.1 TODO-E3-A — codegen_bool_expr(): operadores && e || (codegen.c)

A função `codegen_bool_expr(ctx,expr)` traduz uma expressão booleana e retorna uma `bool_result_t` com duas listas de saltos pendentes.

O caso de expressão atômica (relacional, variável, literal) já está implementado como exemplo. Complete os casos `&&` e `||` usando as funções de `backpatch.h`.

6.7.2 TODO-E3-B — codegen_stmt(): comando if/else (codegen.c)

Substitua o stub do caso `AST_IF` pela implementação completa: 1. Chame `codegen_bool_expr()` para a condição

(stmt->children[0]); 2. Preencha true_list com o rótulo do then-block e emita o rótulo; 3. Gere código do then-block (stmt->children[1]); 4. Se houver else (stmt->children[2]): emita goto _Lend, preencha false_list, emita rótulo, gere código do else-block, emita _Lend; 5. Se não houver else: preencha false_list com o rótulo de saída

6.7.3 TODO-E3-C — codegen_stmt(): comando while (codegen.c)

Substitua o stub do caso AST_WHILE pela implementação completa seguindo o esquema _Lbegin / _Lbody / goto _Lbegin / _Lend.

6.7.4 TODO-E3-D — codegen_stmt(): comando for (codegen.c)

Substitua o stub do caso AST_FOR pela implementação completa. Os filhos do nó AST_FOR são: [0]=init, [1]=cond, [2]=step, [3]=body.

6.8 Aulas Relacionadas

Os tópicos necessários para a Etapa 3 são abordados nas aulas anteriores ao laboratório de 26 de outubro, consistindo em uma preparação direta para esta etapa.

6.9 Testando a Etapa 3

```
cd etapa3/  
make  
bash tests/run_tests.sh
```

Os 6 testes válidos incluem if simples, if/else, while, for, expressão &&, e fatorial recursivo. Cada .l tem um arquivo .expected com o TAC correto.

7 Etapa 4 — Geração de Assembly x86-64

Laboratório: 18 de outubro de 2026

7.1 Objetivo

Transformar o TAC da Etapa 3 em **código assembly x86-64 real e executável**. O pipeline completo do compilador passa a ser:

programaLARA → AST → TAC → assemblyx86-64

O arquivo assembly gerado pode ser montado e executado com:

```
./lara < programa.lc > programa.s  
gcc programa.s -o programa -no-pie  
./programa
```

7.2 Convenções de chamada (System V ABI x86-64)

O compilador segue a ABI padrão do Linux x86-64:

Elemento	Convenção
1º–6º argumento inteiro	%rdi, %rsi, %rdx, %rcx, %r8, %r9
Argumentos adicionais	Pilha (ordem inversa)
Valor de retorno	%rax
Registradores livres	%rax, %rcx, %rdx, %rsi, %rdi, %r8–r11
Registradores salvos	%rbx, %rbp, %r12–r15
Alinhamento do stack	16 bytes antes de call

7.2.1 Prólogo padrão de função

```
fname:  
    pushq    %rbp  
    movq     %rsp, %rbp  
    subq     $N, %rsp ; N = tamanho do frame (múltiplo de 16)
```

7.2.2 Epílogo padrão de função

```
    movq     %rbp, %rsp  
    popq     %rbp  
    ret
```

7.3 Alocação de variáveis e temporários

Classe	Onde fica	Como acessar
Variável global v	Segmento .bss	v(%rip)
Variável local x	Frame, offset calculado na E2	-off(%rbp)
Temporário _tN	Frame, 8 bytes por temporário	-off(%rbp)

7.4 Mapeamento TAC → Assembly

7.4.1 Operações aritméticas

TAC	Assembly
t=a+b	movq a,%rax/ addq b,%rax/ movq %rax,t
t=a-b	movq a,%rax/ subq b,%rax/ movq %rax,t

TAC	Assembly
t=a*b	movq a,%rax/ imulq b/ movq %rax,t
t=a/b	movq a,%rax/ cqto/ idivq b/ movq %rax,t
t=a%b	movq a,%rax/ cqto/ idivq b/ movq %rdx,t

7.4.2 Operadores relacionais

TAC	Assembly
t=a<b	movq a,%rax/ cmpq b,%rax/ setl %al/ movzbq %al,%rax/ movq %rax,t
t=a==b	igual, com sete
t=a!=b	igual, com setne
(outros)	setg, setle, setge, setlconforme o operador

7.4.3 Desvios e rótulos

TAC	Assembly
label L	L:
goto L	jmp L
ifTrue t goto L	cmpq \$0,t/ jne L ifFalse t
goto L cmpq \$0,t/ je L	

7.4.4 Chamada de função

```

; param a1 (já coletado antes)
; param a2
; call f 2
movq a1, %rdi
movq a2, %rsi
call f
movq %rax, result ; se chamada com retorno

```

7.4.5 I/O (printf / scanf)

```

; print v (inteiro)
movq v, %rsi
leaq _fmt_int(%rip),
%rdi xorq %rax, %rax
call printf

```

7.5 Otimizações peephole

O módulo peephole.c aplica substituições locais sobre o assembly gerado (leitura linha a linha, com janela de 2–3 linhas). O padrão exemplo já implementado:

```
movq %rax, X
movq X, %rax
```

← redundante; pode ser removida

Outros padrões que os alunos devem implementar:

Antes	Depois	Descrição
jmpLseguido de L: L:		Salto para o próximo
addq \$1, X	incqX	Incremento
subq \$1, X	decqX	Decremento

7.6 Exemplo end-to-end

```
cd etapa4/
make
./lara < tests/valid/02_fatorial.lc > /tmp/fat.s
gcc /tmp/fat.s -o /tmp/fat -no-pie
/tmp/fat
# saída: 120
```

7.7 Tarefas dos alunos

7.7.1 TODO-E4-A — Expressões aritméticas e relacionais (asmgen.c)

Complete `asmgen_instr()` para os casos `TAC_ADD`, `TAC_SUB`, `TAC_MUL`, `TAC_DIV`, `TAC_MOD` e para os 6 operadores relacionais. Use `asmgen_operand()` para obter a representação em assembly de cada operando.

Os casos `TAC_COPY` e `TAC_NEG` estão implementados como exemplos — siga o mesmo padrão.

7.7.2 TODO-E4-B — Convenção de chamada (asmgen.c)

Complete `asmgen_call()` para: 1. Coletar os argumentos `TAC_PARAM` que precedem a instrução `TAC_CALL` 2. Passar os primeiros 6 argumentos para os registradores `%rdi-%r9` 3. Empilhar argumentos adicionais (se houver) com `pushq` 4. Emitir `call <nome_da_função>` 5. Recuperar o valor de retorno de `%rax` para `result` (se a chamada tem retorno)

7.7.3 TODO-E4-C — Otimizações peephole (peephole.c)

Implemente pelo menos **3 padrões de substituição** além do exemplo fornecido. As funções `pattern_jump_label`, `pattern_add_one_to_inc`, `pattern_sub_one_to_dec` já têm o esqueleto — preencha a lógica de detecção e substituição.

7.7.4 TODO-E4-D — Otimização sobre o TAC (bônus)

Implemente **pelo menos uma** otimização sobre a lista TAC, antes da geração de assembly. Escolha entre:

Otimização	Descrição
Propagação de constantes	$x=5; y=x+3 \rightarrow y=8$
Otimização	Descrição
Folding de constantes	<code>_t1=3*4</code> calculado em tempo de compilação
Eliminação de código morto	Remove instruções cujo resultado nunca é usado

Documente a otimização escolhida e sua abordagem no arquivo `doc/otimizacao.md`.

7.8 Aulas Relacionadas

Os tópicos necessários para a Etapa 4 são abordados nas aulas entre os laboratórios de maio e junho.

7.9 Testando a Etapa 4

```
cd etapa4/  
make test
```

O script compila cada programa `.lc`, monta com `gcc` e executa o binário, comparando a saída com o arquivo `.expected`. Os testes incluem:

Arquivo	Saída esperada
01_soma_exec.lc	7
02_fatorial.lc	120
03_fibonacci.lc	55
04_while_exec.lc	55
05_for_exec.lc	(resultado correto)

8 Notas, Avaliação e Entrega

8.1 Colaboração com GitHub

O projeto é desenvolvido em grupo ao longo de várias semanas — antes, durante e depois de cada sessão de laboratório. Para facilitar a colaboração e garantir que o trabalho não se perde, **recomenda-se fortemente o uso de GitHub**.

O repositório GitHub serve também como **registro de contribuição individual**: o professor poderá acessar o histórico de commits para verificar o que cada integrante do grupo implementou.

8.1.1 Configuração inicial (uma única vez por grupo)

1. Crie uma conta no GitHub (se ainda não tiver): github.com.
2. Crie um repositório privado chamado, por exemplo, lara-compilador: - Acesse github.com/new - Marque **Private** (repositório privado) - Não inicialize com README (o código já existe localmente)
3. Configure o Git localmente (em cada máquina onde o grupo trabalha):

```
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"
```

4. Inicialize o repositório local e faça o primeiro push:

```
# Dentro do diretório raiz do projeto (onde estão etapa1/, etapa2/, ...)
git init
git add etapa1/ # adiciona apenas a etapa atual
git commit -m "Código inicial fornecido - Etapa 1"
git branch -M main
git remote add origin https://github.com/SEU_USUARIO/lara-compilador.git
git push -u origin main
```

5. Adicione os colegas como colaboradores: - No GitHub: Settings → Collaborators → Add people - Cada colega aceita o convite por e-mail

6. Cada colega clona o repositório:

```
git clone https://github.com/SEU_USUARIO/lara-compilador.git
cd lara-compilador
```

8.1.2 Fluxo de trabalho diário

Antes de começar a trabalhar, sincronize com o repositório remoto:

```
git pull # baixa as alterações dos colegas
```

Ao terminar uma sessão de trabalho ou um TODO:

```
git add src/scanner.l # adiciona apenas o(s) arquivo(s)
modificado(s)
git commit -m "TODO-A: suporte a tabs e \r no scanner"
git push # envia para o
GitHub
```

Boas práticas de mensagem de commit:

- Seja específico: "TODO-B: regex FLOAT com expoente" em vez de "correção"
- Um commit por TODO (ou grupo de TODOs pequenos relacionados)
- Todos os integrantes devem ter commits — o professor verifica o histórico

Se dois integrantes editaram o mesmo arquivo:

```
git pull # pode gerar conflito # Resolva o conflito
editando o arquivo manualmente

git add src/parser.y
git commit -m "Merge: integra TODO-C e TODO-D" git push
```

8.1.3 Submissão no Moodle

A submissão no Moodle é um .tar.gz gerado a partir do repositório local após o último git pull:

```
# Dentro do diretório da etapa (ex: etapa1/):
tar czf etapa1-grupo-NOME.tar.gz \
  --exclude='src/*.o' \
  --exclude='src/lex.yy.c' \
  --exclude='src/parser.tab.*' \
  --exclude='lara' \
  src/ tests/ Makefile
```

Verifique o conteúdo antes de enviar:

```
tar tzf etapa1-grupo-NOME.tar.gz
```

O professor pode clonar o repositório GitHub para verificar o histórico de contribuições, independentemente do .tar.gz submetido.

8.2 Forma de entrega e prazos

8.2.1 Sessão de laboratório

No encontro de cada etapa, o grupo demonstra ao professor:

1. Compilação com make sem erros
2. Execução de pelo menos 3 programas de teste (bash tests/run_tests.sh)
3. Compreensão do código escrito (o professor pode perguntar a qualquer integrante)

A avaliação parcial é registrada durante a sessão.

8.2.2 Janela de 48 horas

Após cada laboratório, os grupos têm **48 horas** para incorporar as correções identificadas durante a sessão e submeter a versão final no Moodle.

Etapa	Laboratório	Deadline Moodle
1	26/08	28/08 às 18h
2	30/09	01/10 às 18h
3	26/10	28/10 às 18h
4	18/11	19/11 às 18h

8.2.3 Critério de aceite técnico

O arquivo entregue deve satisfazer:

1. make compila sem erros (warnings são aceitos)
2. ./lara < tests/valid/01_*.lc produz alguma saída sem travar
3. ./lara < tests/invalid/01_*.lc retorna código de saída != 0
4. bash tests/run_tests.sh executa sem erro de script

8.3 Critérios de avaliação

8.3.1 Etapa 1

Critério	Peso
TODO-A: espaço branco com \te \r	10%
TODO-B: float com expoente (1.5e10)	15%
TODO-C: operadores lógicos &&e \ \	15%
TODO-D: lista de argumentos com vírgula	20%
TODO-E: ast_count_nodes com next	10%
TODO-F: ast_count_leaves	15%
TODO-G: ast_max_depth	15%

8.3.2 Etapa 2

Critério	Peso
TODO-E2-A: offsets de variáveis globais	20%
TODO-E2-B: TAC para expressões aritméticas	30%
TODO-E2-C: TAC para operadores relacionais	20%
TODO-E2-D: TAC para atribuição simples :=	30%

8.3.3 Etapa 3

Critério	Peso
TODO-E3-A: booleanos com curto-circuito &&\ \	25%
TODO-E3-B: if/else com back-patching	25%
TODO-E3-C: while com back-patching	25%
TODO-E3-D: for com back-patching	25%

8.3.4 Etapa 4

Critério	Peso
TODO-E4-A: assembly para expressões aritméticas/rel.	30%
TODO-E4-B: convenção de chamada correta	25%
TODO-E4-C: otimizações peephole (mínimo 3 padrões)	20%
TODO-E4-D: otimização sobre o TAC	15%
Testes de execução corretos	10%

8.4 Notas sobre corretude e estilo

Corretude automática. Cada TODO é avaliado pelos testes automatizados fornecidos. O resultado produzido pelo compilador dos alunos deve ser **idêntico** (byte a byte) ao arquivo .expected correspondente, que foi gerado com a solução de referência.

Estilo de código. Soluções com código claro, comentado nos pontos não triviais e sem duplicação desnecessária recebem bônus de até 5% por etapa.

Memória. Vazamentos de memória detectáveis com `valgrind` sem `suppressions` resultam em desconto de até 10%. Use `valgrind--leak-check=full./lra<arq.lc`.

Compilação limpa. O código entregue deve compilar sem erros. Warnings gerados pelo código dos alunos (não pelo código fornecido) resultam em desconto de 5%.

Apresentação. No laboratório, o grupo deve demonstrar ao professor: - A compilação do projeto (`make`) - A execução de pelo menos 3 programas de teste - Que entende o código que escreveu (o professor pode fazer perguntas)

Grupos que não conseguirem demonstrar compreensão do código perdem a nota proporcional ao TODO em questão, mesmo que o código compile e passe nos testes.

8.5 Dicas e armadilhas comuns

Gerenciamento de memória. As funções `codegen_expr()` e `tac_new_temp()` retornam strings alocadas dinamicamente que devem ser liberadas com `free()` após uso. Esquecer o `free()` causa vazamentos que se acumulam em programas grandes.

Temporários em TAC. O módulo `tac.c` mantém um contador global de temporários (`_t1`, `_t2`, ...). Nunca reinicie esse contador durante a compilação de um único programa, pois os nomes devem ser únicos no TAC inteiro.

Precedência no parser. A gramática LARA usa a diretiva `%left` e `%right` do bison para resolver ambiguidades. Se os testes produzirem associações incorretas, verifique a ordem das declarações de precedência em `parser.y`.

Assembly AT&T vs. Intel. O compilador usa a sintaxe AT&T (`movq %rax, %rbx` move `%rax` para `%rbx`, a fonte vem primeiro). Alguns tutoriais usam a sintaxe Intel (operandos invertidos) — não confunda.

-no-pie. A opção `gcc-no-pie` é necessária para que o acesso a variáveis globais via `nome(%rip)` funcione sem relocação dinâmica. Sem ela, o linker pode gerar erros de relocação.

`imulq` com dois operandos. A instrução `imulq src` (um operando) multiplica `%rax` por `src` e coloca o resultado em `%rdx:%rax`. Use apenas `%rax` como resultado. A forma `imulq src, dst` (dois operandos) evita mexer em `%rdx`.

8.6 Recursos e referências

- **Flex manual:** `man flex` ou `flex.sourceforge.net`
- **Bison manual:** `man bison` ou `gnu.org/software/bison`
- **x86-64 ABI:** *System V AMD64 ABI*, disponível em `refspecs.linuxfoundation.org`
- **Instruções x86-64:** Intel Software Developer's Manual, Vol. 2 (instruction reference)
- **Compiladores — livro-texto:** Aho, Lam, Sethi, Ullman — *Compilers: Principles, Techniques, and Tools* (Livro do Dragão), 2ª ed.
 - Capítulo 3: Análise Léxica (Etapa 1)
 - Capítulo 4: Análise Sintática (Etapa 1)
 - Capítulo 6: Geração de Código Intermediário (Etapas 2 e 3)
 - Capítulo 8: Geração de Código (Etapa 4)

– Capítulo 9: Otimizações (Etapa 4, bônus)

Especificação gerada em 19 de agosto de 2026 — INF01083/2026-2 — UFRGS/INF